

Agentic AI Engineering Internship 2026

Week 3 Comprehensive Report

Embeddings, RAG & Vector Databases

Prepared By:
Ghafoor

Date:
July 19, 2026

CalderR.

RETHINKING · HOW · WORK · WORKS

Contents

- 1 Executive Summary** 2
- 2 Conceptual Mastery & Assessment** 2
 - 2.1 Embeddings, RAG & Vector Databases 2
- 3 Production Project: Enterprise Document Intelligence Platform** 3
 - 3.1 Technical Implementation 3
 - 3.2 System Demonstrations 3
- 4 Intermediate Project: Personal Knowledge Base** 5
 - 4.1 Technical Implementation 5
 - 4.2 System Demonstrations 5
- 5 Foundational Labs Deep-Dive** 7
 - 5.1 Lab 3.1: Semantic Search CLI 7
 - 5.2 Lab 3.2: Naive RAG Pipeline 8
 - 5.3 Lab 3.3: Hybrid Retrieval & RAGAS Evaluation 10
- 6 Conclusion & Next Steps** 10

1 Executive Summary

This comprehensive report documents the theoretical learnings, hands-on labs, and advanced architectural implementations completed during Week 3 of the CalderR Agentic AI Engineering Internship. Building upon the prompt engineering and tool-calling mechanics established previously, the core objective of this week was to master Retrieval-Augmented Generation (RAG) at scale. The week introduced the foundational technologies that allow AI systems to reason over external knowledge they were never trained on, focusing heavily on vector embeddings, vector databases, and sophisticated retrieval architectures.

The week culminated in the successful deployment of two complex projects: a **Personal Knowledge Base** (Intermediate) and an **Enterprise Document Intelligence Platform** (Production). These projects synthesized the week's learnings by implementing multi-tenant data ingestion pipelines, hybrid search mechanisms (BM25 + Semantic), cross-encoder re-ranking for enhanced accuracy, and rigorous evaluation using the RAGAS framework.

2 Conceptual Mastery & Assessment

The curriculum emphasized transitioning from naive generation into robust, fact-based AI systems via vector-based retrieval.

2.1 Embeddings, RAG & Vector Databases

- **Embeddings & Semantic Search:** Explored the mathematical representation of textual meaning via high-dimensional vector spaces. Implemented cosine similarity metrics and compared local embedding models (e.g., `sentence-transformers`, `all-MiniLM-L6-v2`, and `bge-small-en`).
- **Vector Databases:** Mastered the storage, indexing, and querying of embeddings. Utilized ChromaDB and FAISS for efficient nearest-neighbor searches, focusing on the tradeoffs between in-memory structures and persistent server-based storage.
- **Advanced RAG Architecture:** Evolved beyond naive RAG (load → split → embed → retrieve → generate) by building sophisticated query pipelines. Implemented Hybrid Retrieval (Semantic + BM25 keyword search) utilizing LangChain's `EnsembleRetriever`, augmented by Cross-Encoder re-ranking to drastically improve top-k retrieval accuracy.
- **RAG Evaluation Frameworks:** Adopted an engineering mindset toward LLM evaluation. Integrated the RAGAS framework to quantitatively measure system performance across key metrics: *faithfulness*, *answer relevancy*, *context precision*, and *context recall*.

3 Production Project: Enterprise Document Intelligence Platform

Architectural Overview

The flagship deliverable for Week 3 is a robust, multi-tenant Enterprise Document Intelligence Platform. Built using FastAPI and Streamlit, it allows users to securely upload, process, and query documents. The backend features asynchronous document processing, tenant-isolated ChromaDB collections, and an advanced RAG pipeline leveraging hybrid search and re-ranking to deliver highly accurate answers accompanied by confidence scores and source citations.

3.1 Technical Implementation

- **Multi-Tenant Async Backend:** Engineered a scalable FastAPI backend with background task workers (`asyncio`) handling heavy document chunking and embedding processes.
- **Tenant Isolation in Vector DB:** Dynamically namespaced ChromaDB collections ensure strict data isolation between different enterprise clients.
- **Advanced Retrieval Pipeline:** Executed a multi-stage retrieval architecture: a base Hybrid Search (BM25 + ChromaDB semantic search) followed by a BAAI `bge-reranker` to reorder the context chunks before LLM synthesis.

3.2 System Demonstrations

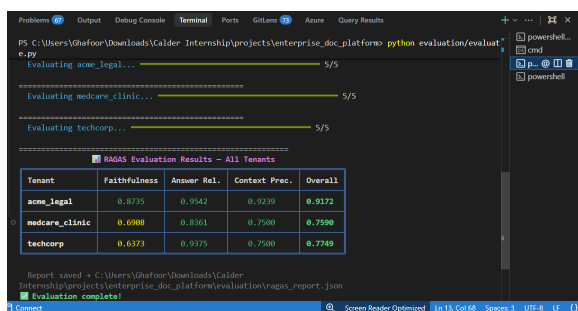


Figure 1: FastAPI Multi-Tenant End-points

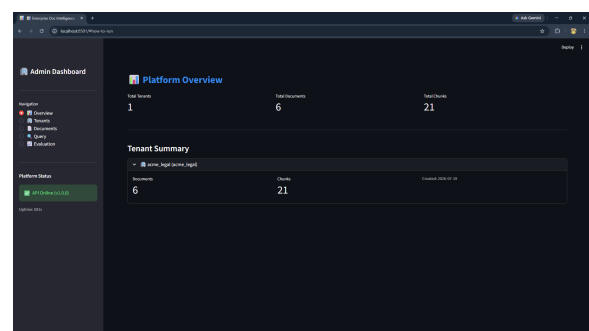


Figure 2: Streamlit Admin Dashboard

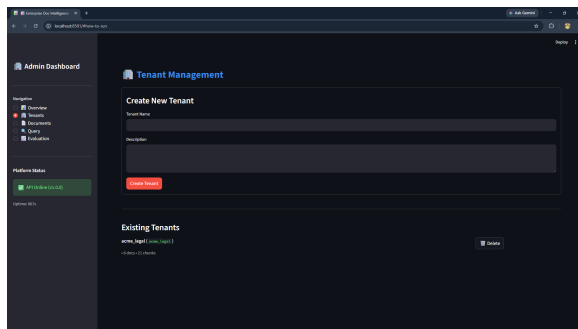


Figure 3: Existing Tenants and Management

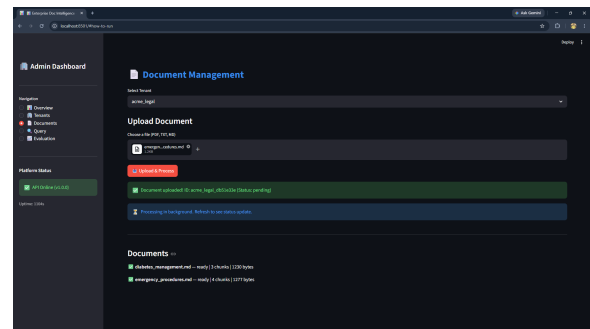


Figure 4: Tenant Document Processing Status

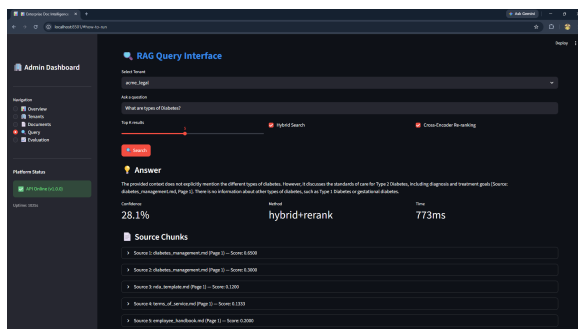


Figure 5: Advanced Query Retrieval Results

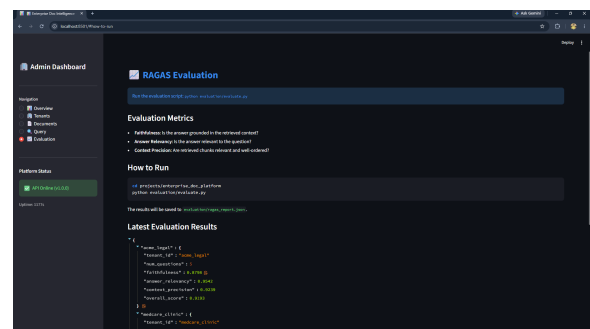


Figure 6: RAGAS Evaluation Commands and Guidelines

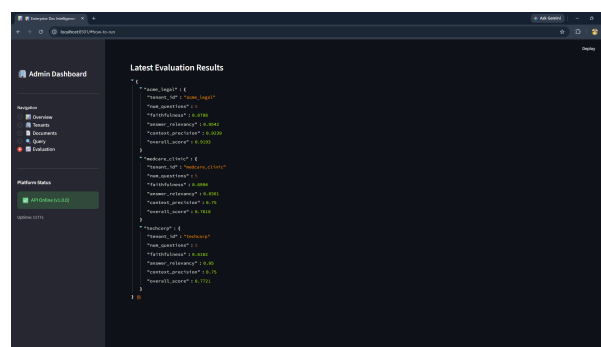


Figure 7: RAGAS Evaluation Scores & Confidence Metrics

4 Intermediate Project: Personal Knowledge Base

Architectural Overview

A CLI-based multi-document Question and Answer system. This agent ingests disparate file formats (PDFs, text files, markdown), chunks the text, and stores the embeddings in a local ChromaDB instance, allowing the user to dynamically query their own customized, local knowledge repository.

4.1 Technical Implementation

- **Multi-Format Document Loaders:** Utilized LangChain loaders to seamlessly ingest structured and unstructured data formats.
- **Chunking & Local Embeddings:** Experimented with RecursiveCharacterTextSplitter and embedded the resulting chunks utilizing local `sentence-transformers` to avoid third-party API costs.
- **LangChain RetrievalQA:** Orchestrated the retrieval and generation phases using LangChain's modular chains, ensuring every generated answer is heavily grounded by citing specific document chunks and metadata.

4.2 System Demonstrations

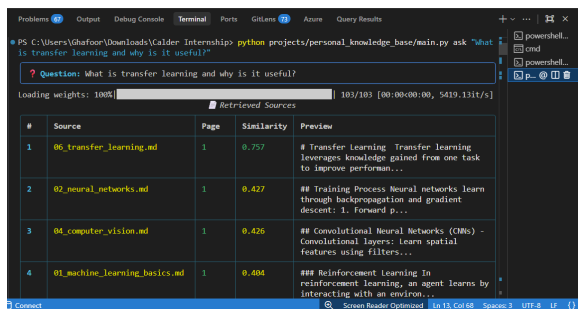


Figure 8: CLI Initialization & Database Setup

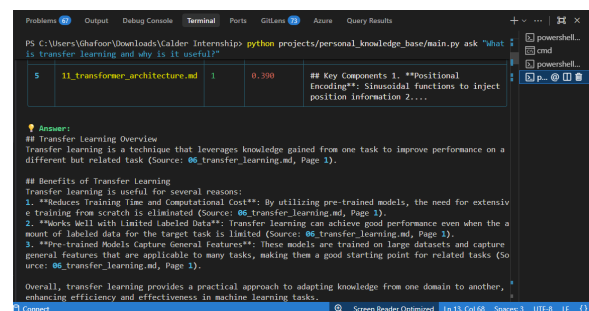


Figure 9: Document Parsing and Chunking

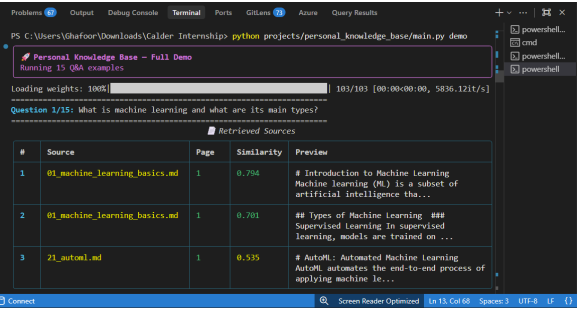


Figure 10: ChromaDB Vector Ingestion

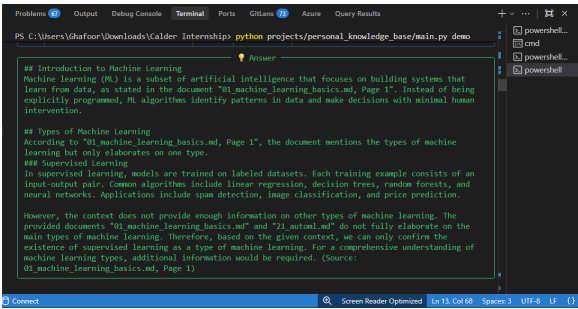


Figure 11: Querying the Local Knowledge Base

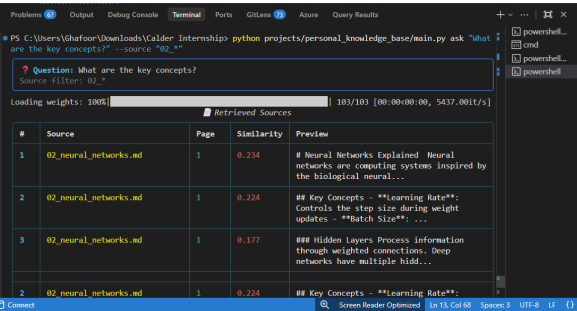


Figure 12: Response Generation & Source Sourcing

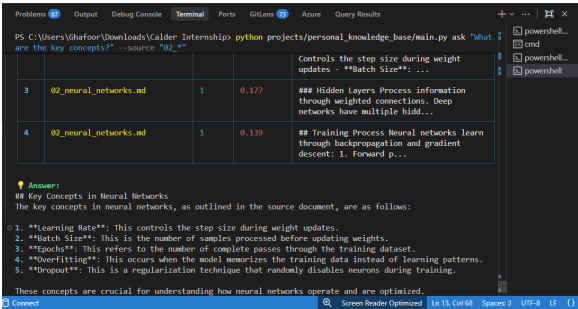


Figure 13: Streaming Responses in the CLI

5 Foundational Labs Deep-Dive

5.1 Lab 3.1: Semantic Search CLI

Focused on the core mathematics of vector similarity and local embedding models.

- **Embedding Spaces:** Computed semantic similarities using Cosine Distance across 100 Wikipedia sentences.
- **Model Comparisons:** Embedded the dataset using two different HuggingFace models (all-MiniLM-L6-v2 vs bge-small-en) and visualized the distribution of vectors using 2D PCA.

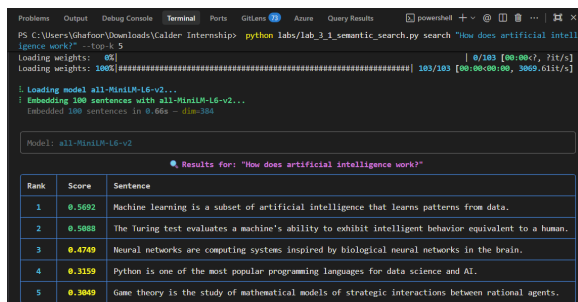


Figure 14: Sentence Embedding Computations

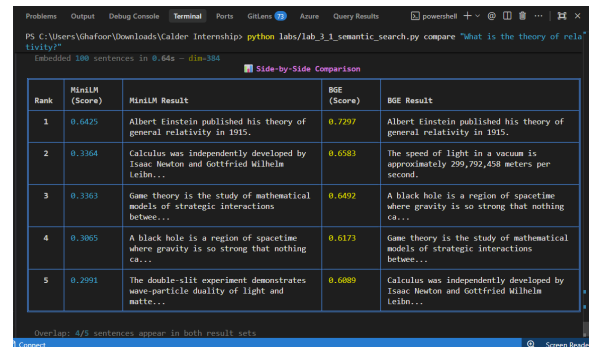


Figure 15: Cosine Similarity Comparisons

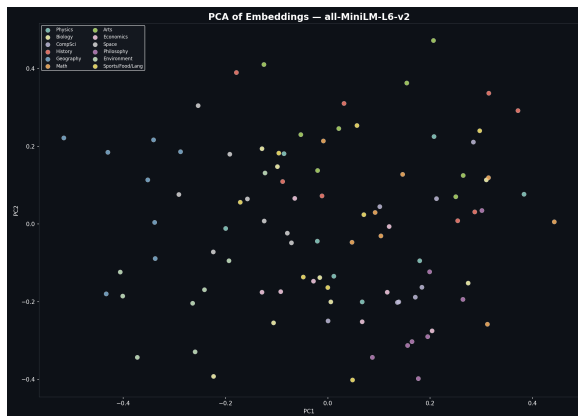


Figure 16: Model 1 Vector Search Results

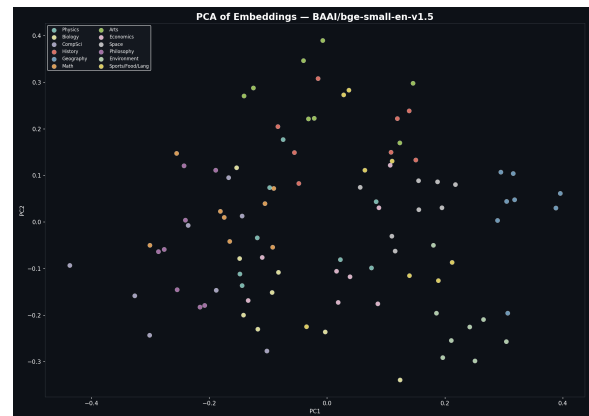


Figure 17: Model 2 Vector Search Results

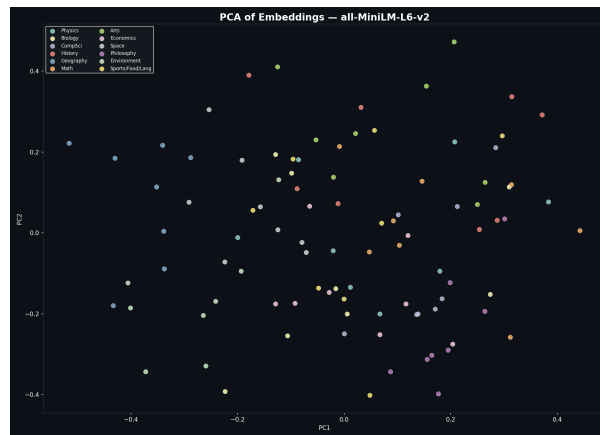


Figure 18: 2D PCA Visualization of Embeddings

5.2 Lab 3.2: Naive RAG Pipeline

Constructed a standard Retrieval-Augmented Generation pipeline to learn baseline architectures.

- **Data Ingestion:** Processed 50 pages of PDF content, evaluating the impact of chunk sizes (256 vs 512 vs 1024 tokens) on context retrieval limits.
- **ChromaDB Persistence:** Set up local vector storage indexing metadata alongside page contents for rudimentary filtering.

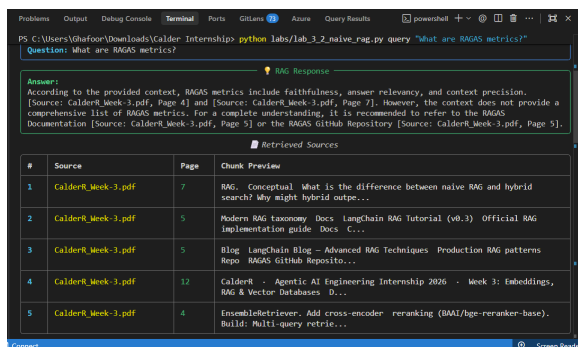


Figure 19: PDF Ingestion and Splitting

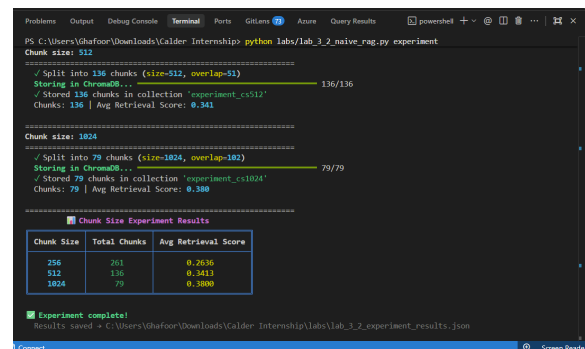


Figure 20: ChromaDB Indexing and Persistence

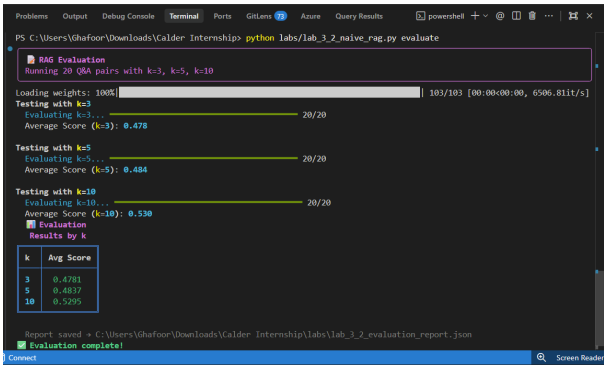


Figure 21: Naive RAG Answer Generation

handling complex external data has been solidified. Furthermore, the integration of RA-GAS provided a vital engineering capability: the ability to empirically measure LLM application performance rather than relying on subjective "vibe checks."

Moving into the final stages of the internship, the focus will shift towards large-scale orchestration, autonomous multi-agent systems, and deploying stateful graphs that combine our knowledge bases with complex tool-calling workflows.